PNFI · Ingeniería del Software

Sección IN3103 · Equipo 8 (Naty's)

Automatización de las Pruebas y Pipelines

# Fundamentos de la Automatización de Pruebas

*Punto investigado y ejemplos aplicados al sistema Naty's*

**Expositor:** Elías Armas

**Integrantes:** Elías Armas · Moisés Pereira · Edgmairys Apóstol · Ismael Torres · Santiago Loyo

**Sistema de referencia:** Sistema de Gestión Inteligente de Pedidos, Créditos y Compras con Análisis Predictivo IA para La Larense (Naty's)

**Fecha de la exposición:** 13 de noviembre de 2026

# Contenido

1. Ubicación del punto en la exposición
2. ¿Qué es la automatización de pruebas?
3. Prueba manual vs. prueba automatizada
4. Beneficios y limitaciones
5. ¿Cuándo conviene automatizar y cuándo no?
6. La pirámide de pruebas
7. Etapas para automatizar las pruebas
8. Lo que la automatización encontró en Naty's
9. Ejemplos para la guía y las diapositivas
10. Conclusión
Referencias

# 1. Ubicación del punto en la exposición

La exposición del Equipo 8, «Automatización de las Pruebas y Pipelines», tiene cinco puntos. Este documento desarrolla el **primero**, que abre la exposición y prepara el terreno para los demás: si el público no entiende qué es automatizar y por qué conviene, no puede entender los pipelines ni la integración continua.

| # | Punto | Integrante |
|---|---|---|
| 1 | **Fundamentos de la Automatización de Pruebas** (este documento) | **Elías Armas** |
| 2 | Arquitectura y Diseño de Scripts de Prueba | Edgmairys Apóstol |
| 3 | Definición y Tipos de Pipeline | Moisés Pereira |
| 4 | Herramientas de Ecosistema | Ismael Torres |
| 5 | Estrategias Avanzadas e Integración Continua (CI/CD) | Santiago Loyo |

**Objetivo del punto:** que el público entienda qué es la automatización de pruebas, en qué se diferencia de la prueba manual, qué se gana con ella, cuándo conviene aplicarla, cómo se reparten las pruebas en la pirámide y cuáles son las etapas para automatizar, todo explicado con lo que el equipo hizo en el sistema Naty's.

**Sistema de referencia:** todos los ejemplos salen de las pruebas reales del sistema Naty's (proyecto natys-larence, hecho en PHP con MySQL, más el microservicio del chatbot en Python). Se usan sobre todo los módulos de **Entregas** y del **Chatbot**, que son los que el expositor probó y documentó.

# 2. ¿Qué es la automatización de pruebas?

La **automatización de pruebas** es el uso de programas (scripts) y herramientas para ejecutar pruebas de software sin que una persona realice cada paso a mano. El programa prepara los datos, ejecuta la acción, **compara el resultado obtenido con el resultado esperado** y genera un reporte que dice qué pasó y qué falló.

No se trata de que la máquina «piense» la prueba: la prueba la diseña una persona. Lo que se automatiza es la **ejecución y la verificación**, que son la parte repetitiva.

## 2.1 Los tres elementos de toda prueba automatizada

| Elemento | Qué es | Ejemplo en Naty's (módulo Entregas) |
|---|---|---|
| **Datos de entrada** | La información con la que se prueba | Una entrega con id 999 que no existe en la base de datos |
| **Acción** | Lo que el script ejecuta sobre el sistema | Pedir al modelo Entrega que la modifique: manejarAccion('modificar', …) |

| Elemento | Qué es | Ejemplo en Naty's (módulo Entregas) |
|---|---|---|
| **Verificación (aserción)** | La comparación entre lo obtenido y lo esperado | Que responda status = false con el mensaje «Entrega no existe» y que haga rollback |

Si la verificación se cumple, la prueba **pasa**; si no se cumple, **falla** y avisa exactamente dónde. Así se ve la prueba real del módulo de Entregas (UT-03):

```
// tests/EntregaTest.php · test_modificar_entrega_inexistente_es_rechazada

// 1. DATOS: la entrega 999 no existe en la base de datos
$stmtMock->method('fetch')->willReturn(false);
$pdoMock->expects($this->once())->method('rollBack');  // debe deshacer
$pdoMock->expects($this->never())->method('commit');   // nunca confirmar

// 2. ACCIÓN: pedir que se modifique
$resultado = $entrega->manejarAccion('modificar',
                json_encode(['id' => 999, ...]));

// 3. VERIFICACIÓN: comparar con lo esperado
$this->assertFalse($resultado['status']);
$this->assertEquals('Entrega no existe', $resultado['msj']);
```

> **Para decirlo en la exposición**
> «Automatizar una prueba es escribirla una vez y poder ejecutarla mil veces, siempre igual y sin cansarse. En Natys, las 7 pruebas unitarias de Entregas se ejecutan en 0,03 segundos.»

### 2.2 Objetivos de automatizar

- **Ahorrar tiempo** en pruebas que se repiten en cada cambio del código.
- **Detectar errores temprano**, cuando corregirlos es más barato.
- **Dar confianza** para modificar el sistema sin miedo a dañar lo que ya funcionaba.
- **Hacer posible lo imposible a mano**, como probar con muchos usuarios a la vez o con cientos de combinaciones de datos.
- **Dejar evidencia**: cada ejecución deja un reporte con fecha, resultado y error exacto.

## 3. Prueba manual vs. prueba automatizada

Las dos son necesarias y se complementan. La automatización no elimina a la persona que prueba: la libera de lo repetitivo para que use su criterio donde hace falta.

| Criterio | Prueba manual | Prueba automatizada |
|---|---|---|
| Velocidad | Lenta: depende de la persona | Rápida: segundos o minutos |

| Criterio | Prueba manual | Prueba automatizada |
|---|---|---|
| Repetición | Cansada; se saltan pasos | Idéntica cada vez |
| Costo inicial | Bajo | Alto: hay que escribir el script |
| Costo a largo plazo | Alto: se paga en cada ejecución | Bajo: ejecutar es casi gratis |
| Criterio humano | Sí: nota detalles visuales y de uso | No: solo verifica lo programado |
| Evidencia | Depende de que alguien anote | Reporte automático de cada ejecución |
| Ideal para | Exploración, usabilidad, casos únicos | Regresión, flujos críticos, muchos datos, seguridad |

> **Ejemplo en Natys**
>
> **A mano:** para comprobar el formulario «Modificar Entrega» hay que iniciar sesión, entrar a Entregas, buscar la entrega, abrir el formulario y revisar que el título, el sector y los pedidos sean correctos. Son varios minutos, y hay que repetirlo después de cada cambio.
>
> **Automatizado (UI-01):** con Selenium IDE se grabó ese recorrido una sola vez. Ahora el script abre el formulario y verifica solo que el título diga «Modificar Entrega», que el sector sea «Oeste» y que aparezcan el cliente «Farmatodo» y el pedido «#16». Se repite con un clic.

# 4. Beneficios y limitaciones

## 4.1 Beneficios

- **Rapidez y repetición:** la suite unitaria de Entregas corre 7 pruebas y 28 aserciones en 0,031 s.
- **Regresión segura:** después de corregir los errores, se volvieron a ejecutar todas las pruebas de Entregas (regresión RG-01) para confirmar que las correcciones no dañaron nada.
- **Encuentra errores reales:** las pruebas automatizadas de Naty's detectaron defectos que nadie había visto (ver sección 8).
- **Cubre lo que no se puede a mano:** las pruebas de seguridad intentaron inyección SQL y acceso sin sesión de forma sistemática.

## 4.2 Limitaciones

- **Requiere inversión inicial:** escribir y preparar los scripts y el entorno (XAMPP, base de datos de prueba, Composer, PHPUnit).
- **Requiere mantenimiento:** si la pantalla o la base de datos cambian, el script también debe cambiar.
- **Solo verifica lo programado:** si nadie escribió la aserción, el error pasa sin ser visto.
- **No juzga la experiencia:** no dice si una pantalla es cómoda o bonita.

> **Ejemplo en Natys**
> Al ejecutar las pruebas del Chatbot apareció una limitación real: algunas pruebas necesitan la respuesta de Gemini, y cuando la cuenta se quedó sin cuota no podían comprobar la respuesta. El equipo no ocultó esto: esas pruebas se marcaron como omitidas y se dejó constancia en el informe.

## 5. ¿Cuándo conviene automatizar y cuándo no?

| Sí conviene automatizar | Ejemplo en Naty's |
|---|---|
| Pruebas de **regresión** que se repiten tras cada cambio | Las 7 pruebas unitarias de Entregas se ejecutan después de cada corrección |
| **Flujos críticos** del negocio | Registrar, modificar y cancelar entregas; inicio de sesión; pedidos |
| Pruebas con **muchos datos** o combinaciones | Registrar una entrega con datos vacíos, inválidos o incompletos |
| Pruebas de **seguridad** repetibles | Pedir una entrega sin sesión (SEG-01) o intentar inyección SQL (SEG-02) |
| Comprobaciones rápidas de que el sistema **enciende** (humo) | Servidor, conexión a MySQL, tabla entregas, modelo y vista (5 verificaciones) |

| No conviene automatizar | Ejemplo en Naty's |
|---|---|
| Pruebas que se harán **una sola vez** | Revisar que la migración inicial de datos de un cliente quedó bien |
| Pantallas que **cambian a diario** | Una vista que todavía se está rediseñando: el script se rompería en cada cambio |
| Pruebas de **usabilidad o apariencia** | Decidir si los colores y el tamaño del logo de Naty's se ven bien |
| Pruebas **exploratorias** | Recorrer el módulo libremente buscando comportamientos raros |

**Regla práctica:** se automatiza lo que es **repetitivo, estable y crítico**. Lo que requiere criterio humano se deja a las personas.

## 6. La pirámide de pruebas

La pirámide de pruebas, propuesta por Mike Cohn, indica **cuántas pruebas de cada tipo** conviene tener. En la base van muchas pruebas pequeñas y rápidas; arriba, pocas pruebas grandes y lentas. Así los errores se detectan pronto y barato.

| Nivel | Cantidad y velocidad | Qué prueba | Ejemplo en Naty's |
|---|---|---|---|
| **Cima: interfaz / extremo a extremo** | Pocas · lentas y frágiles | El sistema completo como lo usa una persona | UI-01 (Selenium IDE): abrir el formulario Modificar Entrega · AC-01 (Codeception): el usuario consulta entregas |
| **Medio: integración y API** | Moderadas · intermedias | Que las piezas funcionen juntas | IT-01: modificar una entrega en la base de datos real · API-01 (Postman): obtener una entrega por ID |
| **Base: unitarias** | Muchas · muy rápidas | Una función o método aislado | EntregaTest.php: 7 pruebas con objetos simulados (mocks), sin tocar la base de datos |

**Para decirlo en la exposición**

«La pirámide nos dice que pongamos muchas pruebas pequeñas y rápidas abajo y pocas pruebas grandes arriba. En Entregas, las 7 unitarias tardan centésimas de segundo; la de interfaz tarda varios segundos y depende de que la pantalla no cambie.»

## 7. Etapas para automatizar las pruebas

Automatizar no es solo «escribir código de prueba». Es un proceso con etapas, que el equipo siguió en cada módulo de Naty's:

| Etapa | Qué se hace | Cómo se hizo en Naty's (Entregas) |
|---|---|---|
| **1. Analizar y seleccionar** | Decidir qué pruebas vale la pena automatizar | Se eligieron las acciones del requisito RF 14: registrar, consultar, modificar, eliminar y obtener entregas |
| **2. Escoger la herramienta** | Elegir según el tipo de prueba | PHPUnit (unitarias, integración y humo), Postman (API y seguridad), Selenium IDE (interfaz) y Codeception (aceptación) |
| **3. Preparar el entorno** | Dejar listo dónde se ejecutan | XAMPP con Apache y MySQL, Composer, PHPUnit y el archivo phpunit.xml |
| **4. Diseñar y escribir los scripts** | Escribir cada caso con datos, acción y verificación | tests/EntregaTest.php, tests/EntregaIntegracionTest.php y el plan de Selenium |
| **5. Ejecutar** | Correr las pruebas | vendor\bin\phpunit tests\EntregaTest.php --testdox |
| **6. Analizar y reportar** | Revisar resultados y registrar defectos | Informe de pruebas del módulo con capturas, resultados y defectos BUG-01 a BUG-03 |

| Etapa | Qué se hace | Cómo se hizo en Naty's (Entregas) |
|---|---|---|
| **7. Mantener** | Ajustar los scripts cuando el sistema cambia y volver a ejecutar | Regresión RG-01 tras corregir los defectos |

## 8. Lo que la automatización encontró en Naty's

El mejor argumento a favor de automatizar es mostrar errores reales que las pruebas detectaron antes de que llegaran al usuario:

| Defecto | Qué pasaba | Prueba que lo detectó | Estado |
|---|---|---|---|
| **BUG-01** (Entregas) | El modelo aceptaba registrar una entrega con datos incompletos y enviaba la inserción a MySQL, que la rechazaba con los errores 1452 y 1048 | UT-01 (PHPUnit) | Corregido |
| **BUG-02 y BUG-03** (Entregas) | Al modificar una entrega fallaba la consulta en la base real (errores 1054 y 1146) y no se conservaban bien los datos | IT-01 (PHPUnit, integración) | Corregidos |
| **BUG-05** (Chatbot) | El filtro de solo lectura dejaba pasar una segunda sentencia peligrosa, por ejemplo «SELECT 1; DROP TABLE clientes» | UT-01 (pytest) | Corregido |
| **BUG-07** (Chatbot) | Sin cuota de Gemini, el asistente tardaba cerca de 40 s y respondía «200 OK» en lugar de un error | RD-02 (prueba de concurrencia) | Corregido |

> **Para decirlo en la exposición**
> «BUG-05 es el ejemplo perfecto: una prueba automatizada intentó engañar al chatbot para que borrara la tabla de clientes, y lo logró. Lo corregimos antes de que pasara de verdad.»

## 9. Ejemplos para la guía y las diapositivas

Esto es lo que pide la profesora: cada idea del punto con el ejemplo que se va a usar. Se puede copiar tal cual a la guía de la exposición y a las diapositivas.

| Idea | Ejemplo que se usa | Diapositiva sugerida |
|---|---|---|
| Qué es automatizar | UT-03: modificar la entrega 999 inexistente → el sistema responde «Entrega no existe» y hace rollback | Código de la prueba con las 3 partes marcadas: datos, acción, verificación |
| Manual vs. automatizada | Revisar a mano el formulario Modificar Entrega vs. UI-01 en Selenium IDE | Tabla comparativa + captura de Selenium IDE |

| Idea | Ejemplo que se usa | Diapositiva sugerida |
|---|---|---|
| Beneficios | 7 pruebas y 28 aserciones en 0,031 s | Captura de la terminal con PHPUnit en verde |
| Cuándo sí y cuándo no | Sí: entregas, login, seguridad. No: colores del logo, pantallas en rediseño | Dos columnas: ✔ / ✘ |
| Pirámide de pruebas | Base: EntregaTest · Medio: IT-01 y Postman · Cima: Selenium y Codeception | Pirámide con las herramientas de Naty's en cada nivel |
| Etapas | Las 7 etapas aplicadas a Entregas (RF 14) | Línea de tiempo con 7 pasos |
| Errores encontrados | BUG-01 (Entregas) y BUG-05 (Chatbot) | Captura del fallo en rojo y luego en verde |

### 9.1 Apertura sugerida (rompehielo)

«Buenos días. Una pregunta antes de empezar: ¿quién ha tenido que probar lo mismo una y otra vez en su proyecto, por ejemplo iniciar sesión diez veces para ver si ya funciona? Levanten la mano… Exactamente. En nuestro sistema, Naty's, comprobar a mano el módulo de Entregas nos tomaba minutos cada vez. Hoy les vamos a mostrar cómo hacer que la computadora haga ese trabajo por nosotros.»

### 9.2 Cierre y transición

«Ya sabemos qué es automatizar, por qué conviene y qué encontramos en Naty's. Ahora Edgmairys nos va a explicar cómo se arma y se diseña un buen script de prueba.»

## 10. Conclusión

La automatización de pruebas permite ejecutar de forma rápida, repetible y documentada las pruebas que antes se hacían a mano. En el sistema Naty's, aplicarla al módulo de Entregas y al Chatbot permitió encontrar y corregir defectos reales, comprobar la seguridad ante accesos sin sesión e inyección SQL, y verificar que las correcciones no dañaron nada. No reemplaza a las personas: les quita lo repetitivo para que dediquen su criterio a lo que una máquina no puede juzgar.

**Idea que el público debe recordar:** «Automatiza lo repetitivo, estable y crítico; deja a las personas lo que requiere criterio».

## Referencias


- Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley. (Origen de la pirámide de pruebas).
- ISTQB. Syllabus Certified Tester Foundation Level y Test Automation Engineering. International Software Testing Qualifications Board.
- Documentación oficial de PHPUnit, Selenium IDE, Postman, Codeception y pytest.

- Equipo 8. Informe de Pruebas · Módulo de Entregas (RF 14) e Informe de Pruebas · Chatbot Natys. Sistema Naty's, 2026.
- Equipo 8. Especificación de Requisitos de Software (SRS) del sistema Naty's, julio 2026.